Practical Cyber Deception — Introduction to “Chaotic Good”

Koifsec Follow 8 min read · Apr 30, 2025

--

Ok, so before we start, I personally think this is a bit of an “offside” topic. While the tech exists, personally I am not familiar with orgs who actually implement deception in a serious manner, nor do I see discussions about this topic on the various social medias or blogs. Is this perhaps more common than I think it is?

To be honest, I am not too surprised about this. Organizations can barely keep up with their “main” security controls and tools, so expecting a “side-quest” subject such as deception to gain in popularity is not very reasonable. With that being said…

In cybersecurity, we spend most of our time reacting: patching, blocking, detecting. But what if we could **actively mess with attackers** instead?

That's the art of **cyber deception** — deliberately feeding attackers wrong information, wasting their time, setting traps, and forcing them into mistakes.

In this post, we'll explore **real, tactical deception techniques** you can implement right now. No million-dollar honeynet farms. No AI buzzwords. Just smart, hands-on tricks that turn your environment into a minefield for intruders.

What is Cyber Deception?

Cyber deception isn't just honeypots sitting in the corner of your network. It's **controlling the attacker's perception of your environment**:

- Misleading reconnaissance
- Setting traps at key points
- Creating believable artifacts that waste their time
- Turning detection from passive to *aggressive observation*

Done right, deception **increases attacker cost** and **buys defenders time**. I also want to say this at the beginning — this is NOT a solution that should replace traditional security controls, but to ENHANCE them. **If your "base" is not solid, anything you build on top of it will have shaky foundations.**

The interaction of the attacker with the environment

When an attacker (or red team) gains access to a system in your network, their first objective is to most likely gather detailed information about that system. This process typically starts with reconnaissance, which involves

assessing the system to understand its security measures and how it operates. They focus primarily on:

- Local scripts that might contain sensitive information or credentials.
- AV / EDR Reconnaissance (I've seen a LOT of this done via WMIC)
- They explore potential vulnerabilities that could be exploited to gain higher privileges or establish persistence on the system.

In essence, their goal is to identify weaknesses and opportunities that can be exploited to further infiltrate the network or maintain access undetected[1].

One of the most effective methods we have in our arsenal, is the source code of the tools that the attacker might use. For example, one of the most popular tools out there is SharpUp. If we start to dig around further into the capabilities of these tools, we can see exactly what files they're looking for:

```
namespace SharpUp.Checks
{
    public class UnattendedInstallFiles : VulnerabilityCheck
    {
        private static string _windir = System.Environment.GetEnvironmentVariable("windir");
        private static string[] _searchLocations =
                {
                    String.Format("{0}\\sysprep\\sysprep.xml", _windir),
                    String.Format("{0}\\sysprep\\sysprep.inf", _windir),
                    String.Format("{0}\\sysprep.inf", _windir),
                    String.Format("{0}\\Panther\\Unattended.xml", _windir),
                    String.Format("{0}\\Panther\\Unattend.xml", _windir),
                    String.Format("{0}\\Panther\\Unattend\\Unattend.xml", _windir),
                    String.Format("{0}\\Panther\\Unattend\\Unattended.xml", _windir),
                    String.Format("{0}\\System32\\Sysprep\\unattend.xml", _windir),
                    String.Format("{0}\\System32\\Sysprep\\Panther\\unattend.xml", _windir)
```

https://github.com/GhostPack/SharpUp/blob/8a7579174ed447181835cb82bf2ec0279e9110d0/SharpUp/Checks/UnattendedInstallFiles.cs#L14

```
var keys = ThisRunTime.GetSubkeyNames(RegistryHive.LocalMachine, "Software\\Policies\\Microsoft\\Windows\\SrpV2") ?? new string[] { };

if (keys != null && keys.Length != 0)
{
    foreach (var key in keys)
    {
        var keyName = key;
        var enforcementMode = ThisRunTime.GetDwordValue(RegistryHive.LocalMachine, $"Software\\Policies\\Microsoft\\Windows\\SrpV2\\{key}", "EnforcementMode"
        var enforcementModeStr = enforcementMode switch
        {
            null => "not configured",
            0 => "Audit Mode",
            1 => "Enforce Mode",
            _ => $"Unknown value {enforcementMode}"
        };

        var ids = ThisRunTime.GetSubkeyNames(RegistryHive.LocalMachine, "Software\\Policies\\Microsoft\\Windows\\SrpV2\\" + key);

        foreach (var id in ids)
        {
            var rule = ThisRunTime.GetStringValue(RegistryHive.LocalMachine, $"Software\\Policies\\Microsoft\\Windows\\SrpV2\\{key}\\{id}", "Value");
            rules.Add(rule);
        }
```

https://github.com/GhostPack/Seatbelt/blob/392171df84472591d4eae7ebd5b1cdc96ba91377/Seatbelt/Commands/Windows/AppLockerCommand.cs#L40

```
$PuTTYPathEnding = "\SOFTWARE\SimonTatham\PuTTY\Sessions"
$WinSCPPathEnding = "\SOFTWARE\Martin Prikryl\WinSCP 2\Sessions"
$RDPPathEnding = "\SOFTWARE\Microsoft\Terminal Server Client\Servers"
```

https://github.com/Arvanaghi/SessionGopher/blob/master/SessionGopher.ps1#L88

Therefore, we can use the above to create detections that will alert on event ID 4663, where the operation is “read”, where the object name contains any of these files or registry keys, and when the process name is unusual. This can also be expanded to the cloud! fake keys, accounts, groups and files — the concept is the same, to alert about an interaction with them.

Introduction to SACLs

SACLs (System Access Control Lists) are part of Windows security descriptors. They don’t control *who can access* an object — they control

Open in app ↗ Sign up Sign in

Medium Search Write

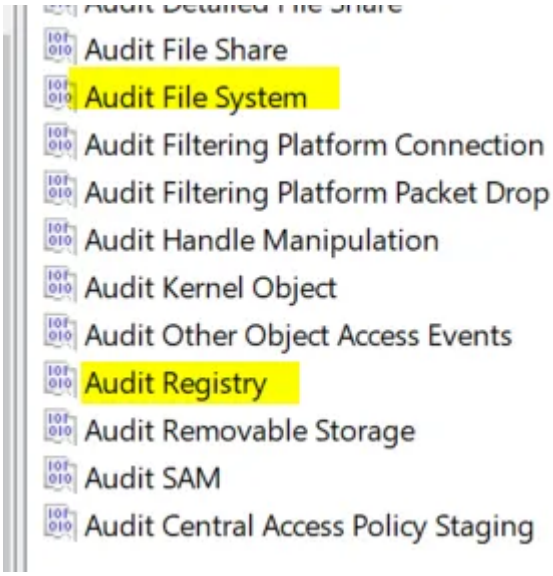
You can use SACLs to **silently log access** to files, folders, registry keys, or objects. Auditing files or registry keys or any other auditable object is a two-step process:

- We first need to enable the capability of audit logging in general
- We then need to enable the auditing itself on our objects of interest

In order to enable the auditing itself, we will head to the group policy editor and enable the following:

- Audit File System -> success and failure
- Audit Registry -> success and failure

These policies are found under '*Computer Configuration*' -> '*Policies*' -> '*Windows Settings*' -> '*Security Settings*' -> '*Advanced Audit Policy Configuration*' -> '*Object Access*':

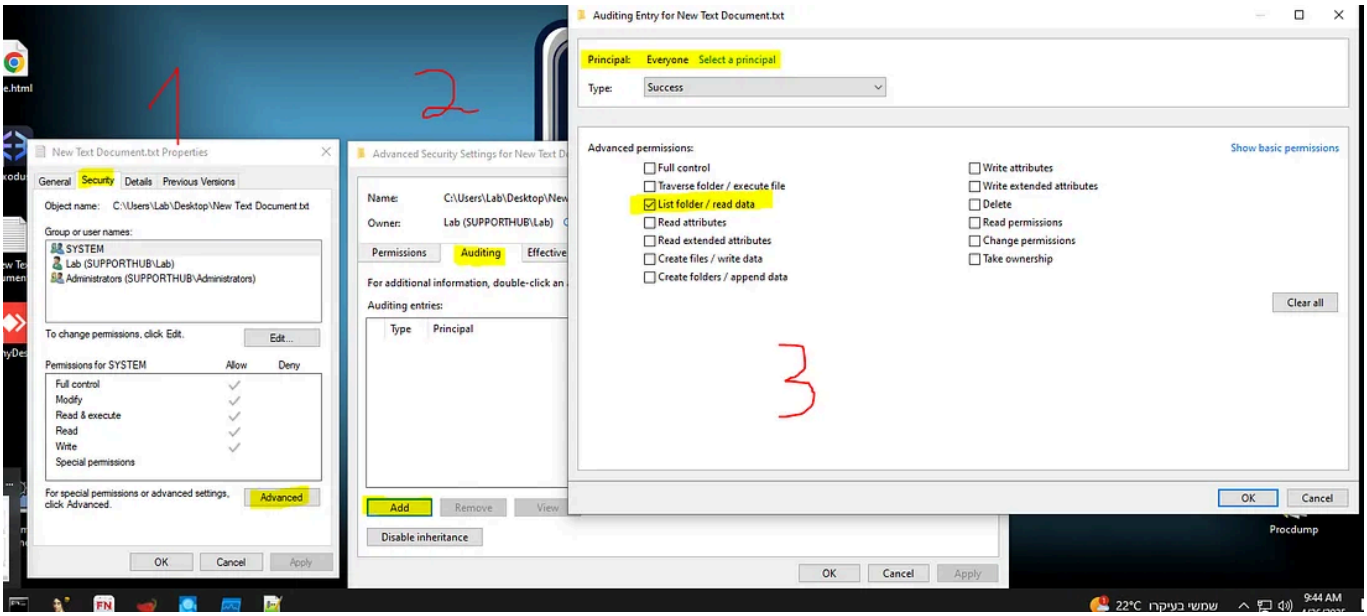


To verify the audit policy:

```
auditpol /get /category:"object access"
```

```
C:\WINDOWS\system32>auditpol /get /category:"object access"
System audit policy
Category/Subcategory                        Setting
Object Access
  File System                               Success
  Registry                                  Success and Failure
```

Now, to audit a specific file or registry key. We can do this by right clicking on the file/folder/key -> properties -> security -> advanced. Then click the “auditing” tab, click “add”, and the select the principle as “Everyone”. Then, configure which actions should be audited. For simplicity I will choose “read”:



Then, I will open the text file, and we will see the following in the event log:

Security Number of events: 28,900

Filtered: Log: Security; Source: ; Event ID: 4663. Number of events: 74

Keywor...	Date and Time	Source	Event ID	Task Category
Audi...	4/26/2025 9:46:59 AM	Micros...	4663	File System
Audi...	4/26/2025 9:39:10 AM	Micros...	4663	Kernel Object

Event 4663, Microsoft Windows security auditing.

General Details

An attempt was made to access an object.

Subject:
Security ID: SUPPORTHUB\Lab
Account Name: Lab
Account Domain: SUPPORTHUB
Logon ID: 0x64185

Object:
Object Server: Security
Object Type: File
Object Name: C:\Users\Lab\Desktop\New Text Document.txt
Handle ID: 0x3e4
Resource Attributes: S:AI

Process Information:
Process ID: 0xb5c
Process Name: C:\Windows\System32\notepad.exe

Access Request Information:
Accesses: ReadData (or ListDirectory)

Access Mask: 0x1

So now that we know how to properly configure auditing, we can use this method to audit interesting files or registry keys for access. This list is truly too long to post, but the way I determine it, is by reading a lot of threat reports and seeing which keys/files are the most common for attackers to query or modify, and develop it from there. SIGMA of course will have good examples too that you can easily search. Here are some good places to start at, based on Outflank's article:

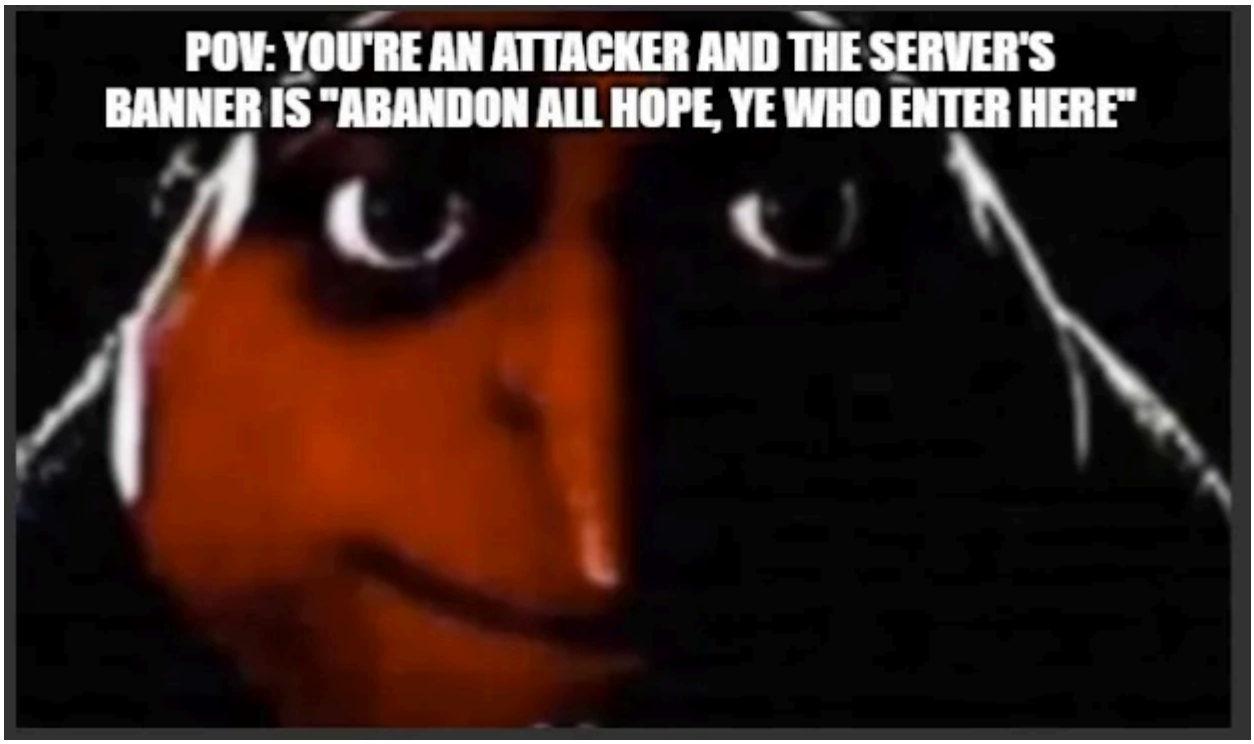
- **AppLocker configuration**. Any read access to the registry key *HKEY_LOCAL_MACHINE\SOFTWARE\Policies\Microsoft\Windows\SrpV2*, from an unexpected process.
- **Saved RDP sessions**. RDP sessions can be stored in the local user's registry. Any read access to the registry key *HKEY_CURRENT_USER\Software\Microsoft\Terminal Server Client*, from an unexpected process.

- **Interesting looking (dummy) script.** If you have a local scripts folder on workstations, add a dummy script with a juicy name like *resetAdminpassword.ps1* and monitor for read access. Make sure the file has some dummy content. Content of this file should not be able to influence the environment.
- **Browser credential stores**. Any read access to browser credential stores such as of Chrome or Firefox, that does not originate from the browser process. If you do not use a certain browser in your environment, planing fake browser credential stores in user's roaming profiles can also be used as a trigger.
- **Other credential stores/private keys.** Any read access to SSH private keys, Azure session cookie files, or password databases that are used by users, that does not originates from an unexpected process.
- **Sysmon configuration file**. Any read access to the Sysmon configuration file of the local system, that does not originate from the Sysmon process.
- **Windows installation/configuration artefacts**. Any read access to *Unattend.xml* in *C:\Windows\Panther*, as covered in this blog post.
- **Interesting (dummy) files on NETLOGON**. Any read access to a file that looks like a legacy script or configuration file, located on the domain controller's NETLOGON share.

If you're running Microsoft's stack, they also have a good article on how to configure deception rules on Defender: https://techcommunity.microsoft.com/blog/microsoftdefenderatpblog/ignite-news-augment-your-edr-with-deception-tactics-to-catch-adversaries-early/3982253

Making attackers / red teamers seethe, some practical examples

Server Name Impersonation



Concept: Attackers often fingerprint servers to choose their attacks (e.g., what web server version is running). By **spoofing the server banner**, you can mislead them.

Example:

- Change your `Apache` server to report `Microsoft IIS 6.0`
- Trick them into using old IIS exploits against your hardened Apache server
- If possible, also tweak error pages and response headers to deepen the illusion
- Slow them down, log their mistakes, and:

```
#######################HELLO########################### ###YOU
ARE LUCKY I SAVED YOU FROM BECOMING A VICTIM###
#################OF ASSHOLE MAL;W!RE##################
################clean your device NOW#################
############DIARHEA TO ALL COINSTEALERS###############
#############EARN YOUR MONEY YOU PUPPETS##############
```

write a funny post about it

Zip Bombs

Concept:
Plant maliciously compressed files that expand to ridiculous sizes when opened.

Example:

- Create a zip bomb that inflates to huge sizes
- Leave it in a juicy-looking share (`\\HR-Server\Compensation2025.zip`)
- **Important:** Remove unzipping tools from your endpoints, so attackers have to exfiltrate and **detonate it *on their own infrastructure***.

Alphabetical Drive Traps for Ransomware

Concept:
Ransomware will sometimes enumerate drives **in alphabetical order** (A:, B:, C:, etc.).

Example:

- Create **A:/B: drive** filled with **junk decoy data**.
- Monitor for any writes, reads, or encryption attempts on B:
  → Instant ransomware early warning system.

Open Ports

Concept:
Attackers/red team will most likely scan your environments for interesting ports or services.

Example:

- Open every possible port on a "lure" server and host fake/empty services for them. Combine with the fake banners for maximum trolling. Make this server actually blend in by making it generate esoteric traffic, but have as little interaction TO it as possible, so that when it alerts, it's high fidelity.

Manipulation SSH

From this Reddit thread:

"One of our guys had the idea to move SSH off port 22 and replace it with a netcat listener that would then just dump /dev/random back to their console, locking it up for a bit.

We also set up SSH Tarpits (acts like ssh and acts like a valid login, but just spins instead of giving an actual shell) across multiple ports as well as the normal SSH Honeypots. It was a good time. Dont think the red team appreciated it though."

Canary Tokens

Concept:
Canary tokens are files, URLs, credentials, etc. that alert you the moment someone touches them. In the context of files, don't forget to turn on auditing for them!

Example:

- Deploy fake AWS keys across your developer share

- Plant DOCX files in random user folders that beacon when opened
- Bury API keys in environment variables that ping you when used
- There are many more possibilities. Visit https://canarytokens.org/nest/ to explore them all. Some ideas:
  — Fake emails/email folders/mailboxes
  — Fake file shares or SharePoint sites (this one is great)
  — Fake documents in legitimate locations
  — Databases with fake tables holding enticing data
  — Fake hidden folders on users' desktops (ones they wouldn't access)
  — Alert on access of files and registry paths often probed by attackers using privilege escalation scripts (what we spoke about at the beginning)
  — Create a "kerberoastable" service account and alert on its use

Summary

Cyber deception isn't about building expensive honeynets or deploying complex traps — it's about **instilling doubt and confusion in the attacker**. By layering practical, tactical deception into your environment, you shift the balance of power: slowing them down, forcing mistakes, and gaining early warning long before real damage is done. From fake servers and canary tokens to ransomware drive traps, deception turns defense from a reactive grind into a strategic, active game.
In short: **stop just defending your network — start *messing with theirs*.**

If you enjoyed the article, feel free to connect with me!
https://www.linkedin.com/in/daniel-koifman-61072218b/
https://x.com/KoifSec
https://detections.xyz/

Red Team Blue Team Cybersecurity

Published in Detect FYI

Follow

1.8K followers · Last published 5 days ago

Threat Detection Engineering and DFIR Insights

Written by Koifsec

Follow

164 followers · 41 following

#DetectionEngineering https://www.linkedin.com/in/daniel-koifman-61072218b/ https://x.com/KoifSec https://detections.xyz/

No responses yet